DSS Application Design Document

Group 198: Parker Conley

Full Project Submission – October 19, 2024

Contents

1 Overview

This document outlines the design of the distributed storage system (DSS). A manager process coordinates and disks, concurrency and state-management decisions, and supporting evidence (diagrams, screenshots, and validation artifacts) required for the project submission.

2 Messaging Envelope

All messages reuse a shared JSON envelope. Individual commands define their own bodies and semantics.

Envelope

```
{
  "version": 1,
  "message_id": "<role>-<pid>-<timestamp_ms>-<counter>",
  "message_type": "<command_or_response>",
  "status_code": "SUCCESS | FAILURE"   // responses only
  "reason": "Optional explanation on FAILURE",
  "body": { /* command-specific fields */ }
}
```

Message ID Format
Pattern `<role>-<pid>-<timestamp_ms>-<counter>` (e.g., `user1-4123-1732665001123-00042`).

- **Role**: Logical sender (e.g., `manager`, `user1`, `diskF`)
- **PID**: Process ID of the sender
- **Timestamp**: Milliseconds since epoch
- **Counter**: Zero-padded integer incremented per message

Purpose: match requests to responses, support retry logic on UDP, deduplicate packets, and produce traceable log entries.

3 Manager Protocol

The manager listens on a single UDP port and accepts nine commands. Responses always mirror the request `message_id`. Every handler performs input validation and returns `status_code` + `reason` on failure. This section summarizes the request/response bodies and the most important error paths; diagrams in Section 8 visualize happy paths.

3.1 Command: `register_user`

Purpose Register a user process and reserve its management/command ports.

Request

```
{
  "version": 1,
  "message_id": "Alice-4123-1732665001123-00001",
  "message_type": "register_user",
  "body": {
```

```
    "user_name": "Alice",
    "ipv4_address": "192.0.2.10",
    "management_port": 21213,
    "command_port": 21214
  }
}
```

Successful Response

```
{
  "version": 1,
  "message_id": "Alice-4123-1732665001123-00001",
  "message_type": "register_user_response",
  "status_code": "SUCCESS",
  "reason": null
}
```

Common Failure Reasons

- User name already registered or not alphabetic (`reason`: `"user_name must contain only alphabetic characters"`).
- Ports outside the group range or already claimed (`reason`: `"management_port must be in range 21200-21299"`).
- Malformed JSON or missing body fields.

3.2 Command: `register_disk`

Purpose Register a disk process and mark it available for assignment.

Request

```
{
  "version": 1,
  "message_id": "DiskC-6123-1732665001567-00001",
  "message_type": "register_disk",
  "body": {
    "disk_name": "DiskC",
    "ipv4_address": "192.0.2.11",
    "management_port": 21205,
    "command_port": 21206
  }
}
```

Successful Response

```
{
  "version": 1,
  "message_id": "DiskC-6123-1732665001567-00001",
  "message_type": "register_disk_response",
  "status_code": "SUCCESS",
```

```
  "body": {
    "state": "Free"
  }
}
```

Common Failure Reasons
- Disk name already in use or not alphabetic.
- Management/command ports invalid or duplicate.
- JSON parsing/validation errors identical to `register_user`.

3.3 Command: `configure_dss`

Purpose Create a new DSS by selecting n free disks and reserving them for the requesting user.

Request

```
{
  "version": 1,
  "message_id": "Alice-4123-1732665002001-00002",
  "message_type": "configure_dss",
  "body": {
    "user_name": "Alice",
    "dss_name": "FirstDSS",
    "n": 3,
    "striping_unit": 1024
  }
}
```

Successful Response

```
{
  "version": 1,
  "message_id": "Alice-4123-1732665002001-00002",
  "message_type": "configure_dss_response",
  "status_code": "SUCCESS",
  "body": {
    "dss_name": "FirstDSS",
    "n": 3,
    "striping_unit": 1024,
    "disks": ["DiskD", "DiskE", "DiskF"]
  }
}
```

Common Failure Reasons
- Requested user is not registered.
- $n < 3$, striping unit not a power of two between 128 B and 1 MiB, or insufficient free disks.
- DSS name already exists.

3.4 Command: `ls`

Purpose Enumerate every configured DSS and the files the manager knows about.

Request {"message_type": "ls", "body": {}}.

Successful Response (truncated example)

```
{
  "message_type": "ls_response",
  "status_code": "SUCCESS",
  "body": {
    "dss_list": [
      {
        "dss_name": "FirstDSS",
        "n": 3,
        "striping_unit": 1024,
        "disks": ["DiskD", "DiskE", "DiskF"],
        "files": [
          {"file_name": "day-of-affirmation.txt",
           "file_size": 392,
           "owner": "Alice"}
        ]
      }
    ]
  }
}
```

Common Failure Reasons If no DSS is configured the manager responds with `status_code="FAILURE"` and `reason="No DSS is configured"`.

3.5 Command: `copy`

Purpose Begin a two-phase copy operation. Phase 1 returns the DSS parameters and acquires a critical section so only one copy per DSS runs at a time.

Request

```
{
  "message_type": "copy",
  "body": {
    "file_name": "wizard-of-oz.txt",
    "file_size": 1238,
    "owner": "Alice"
  }
}
```

Successful Response

```
{
  "message_type": "copy_response",
  "status_code": "SUCCESS",
  "body": {
    "dss_name": "FirstDSS",
    "n": 3,
    "striping_unit": 1024,
    "disks": [
      {"disk_name": "DiskD", "ipv4_address": "192.0.2.21", "command_port": 21208},
      {"disk_name": "DiskE", "ipv4_address": "192.0.2.22", "command_port": 21210},
      {"disk_name": "DiskF", "ipv4_address": "192.0.2.23", "command_port": 21212}
    ]
  }
}
```

Common Failure Reasons

- No DSS configured yet.
- Owner not registered.
- Another copy already in progress for the target DSS (`reason`: `"Copy operation already in progress for DSS 'FirstDSS'"`). The handler uses `self.in_progress_copy` to enforce this critical section.

3.6 Command: `copy_complete`

Purpose Complete the copy transaction by recording the file metadata and releasing the critical section.

Request

```
{
  "message_type": "copy_complete",
  "body": {
    "dss_name": "FirstDSS",
    "file_name": "wizard-of-oz.txt",
    "file_size": 1238,
    "owner": "Alice"
  }
}
```

Successful Response `status_code="SUCCESS"` with no body.

Common Failure Reasons

- No copy currently in progress for the named DSS (e.g., counterparty crashed before phase 2).
- Missing or malformed fields.

3.7 Command: `read`

Purpose Provide the caller with all metadata required to read a file from a DSS.

Request

```
{
  "message_type": "read",
  "body": {
    "dss_name": "FirstDSS",
    "file_name": "wizard-of-oz.txt",
    "user_name": "Alice"
  }
}
```

Successful Response

```
{
  "message_type": "read_response",
  "status_code": "SUCCESS",
  "body": {
    "dss_name": "FirstDSS",
    "file_name": "wizard-of-oz.txt",
    "file_size": 1238,
    "owner": "Alice",
    "n": 3,
    "striping_unit": 1024,
    "disks": [
      {"disk_name": "DiskD", "ipv4_address": "192.0.2.21", "command_port": 21208},
      {"disk_name": "DiskE", "ipv4_address": "192.0.2.22", "command_port": 21210},
      {"disk_name": "DiskF", "ipv4_address": "192.0.2.23", "command_port": 21212}
    ]
  }
}
```

Common Failure Reasons

- DSS does not exist.
- File absent from the DSS catalog.
- Ownership mismatch (`reason`: `"Permission denied: file 'wizard-of-oz.txt' is owned by 'Alice', not 'Bob'"`). This guards cross-user reads.

3.8 Command: `deregister_user`

Purpose Remove a user and free its claimed ports once all DSS operations are complete.

Request `{"message_type": "deregister_user", "body": {"user_name": "Alice"}}`.

Successful Response `status_code="SUCCESS"`; the manager deletes the record and clears both ports from `claimed_ports`.

Common Failure Reasons Attempting to deregister a user that was never registered or providing invalid JSON.

3.9 Command: `deregister_disk`

Purpose Remove a disk when it is no longer part of a DSS.

Request {`"message_type": "deregister_disk", "body": {"disk_name": "DiskD"}`}.

Successful Response `status_code="SUCCESS"`; the disk is removed from the catalog and its ports are cleared.

Common Failure Reasons

- Disk never registered.
- Disk still belongs to an active DSS (`state != "Free"`), resulting in `reason="disk 'DiskD' is currently part of DSS 'FirstDSS'"`.

4 Peer-to-Peer Messaging Between Users and Disks

User processes communicate directly with disk command ports during copy, read, failure, and recovery workflows. Requests are sent as newline-delimited JSON, responses include the shared envelope fields, and all binary payloads are base64-encoded.

4.1 `write_block` (User to Disk)

Purpose Persist a single data or parity block while copying a file.

```
{
  "message_type": "write_block",
  "body": {
    "dss_name": "FirstDSS",
    "file_name": "wizard-of-oz.txt",
    "file_size": 1238,
    "owner": "Alice",
    "stripe_number": 0,
    "block_type": "data",
    "block_data_base64": "<...>"
  }
}
```

Disks reply with `write_block_response` and either `SUCCESS` or an explanatory failure such as `"Block size mismatch"` or `"Disk full"`. Each stripe launches n threads (one per disk), enabling parallel writes.

4.2 `read_block` (User to Disk)

Purpose Fetch a block during a read operation. Successful responses include `block_type` and `block_data_base64`. The user decodes the payload, optionally injects a single-bit error based on the configured probability, and validates parity before accepting the stripe.

4.3 `fail` (User to Disk)

Purpose Simulate a disk crash during the failure-and-recovery scenario. The disk deletes every block in the named DSS and reports the number of removed blocks via `blocks_deleted`.

4.4 `read_block_for_recovery` (User to Surviving Disks)

Purpose Read surviving blocks for a given stripe when rebuilding a failed disk. The request structure matches `read_block`; the semantics remind the disk that the call is part of recovery logging.

4.5 `write_recovered_block` (User to Failed Disk)

Purpose Write reconstructed data or parity back to the failed disk. The body mirrors `write_block` except the `block_type` is derived from the parity-rotation algorithm during recovery.

4.6 `delete_all` (User to Disk)

Purpose Instruct disks to purge all blocks for a DSS while decommissioning. Responses return both `blocks_deleted` and `files_deleted`.

4.7 Threading, Timeouts, and Reliability

- The user client binds a UDP socket to its command port and spawns n threads per stripe for reads and writes. A thread-safe list collects outcomes before parity validation.
- Each P2P exchange uses the same timeout and retry configuration as manager messages. Retries default to zero, but the CLI flags can raise them to recover from packet loss.
- Binary blocks are base64-encoded to keep messages UTF-8 safe. Padding with `0x00` octets ensures every block is exactly `striping_unit` bytes prior to encoding.
- Parity is computed with `compute_xor_parity` and rotates via `parity_disk = n - ((stripe_number % n) + 1)`; both routines live in `user.py`.
- During reads the user injects a random bit flip with probability p (0–100%). If parity fails the entire stripe is re-read, allowing the XOR check to confirm correctness.

5 State Management and Data Structures

Robust state tracking is required across the manager, disks, and users to support concurrent operations and recovery.

5.1 Manager State

- `users`: maps user name to management/command ports and IP address.
- `disks`: records disk metadata, including current state (`Free` vs. `InDSS`) and ownership.
- `dss_catalog`: stores striping parameters and disk assignments for each DSS.
- `files`: maps `(dss_name, file_name)` to size and owner; populated after `copy_complete`.
- `in_progress_copy`: protects each DSS with a single copy critical section. The manager refuses additional `copy` requests until `copy_complete` clears the value.
- `claimed_ports` and `disk_membership`: prevent port reuse and track which disks belong to which DSS for deregistration checks.

All handlers validate protocol version, message IDs, and JSON types before modifying state. Failure paths always return `status_code="FAILURE"` with a descriptive `reason`.

5.2 Disk Storage

Each disk instantiates a `DiskStorage` object (see `disk_storage.py`). The class provides thread-safe methods backed by a single lock:

- `blocks`: dictionary keyed by `(dss_name, file_name, stripe_number)` storing immutable `BlockRecord` instances.
- `files`: dictionary keyed by `(dss_name, file_name)` returning `FileMetadata` records that summarize file ownership, size, and per-disk stripe count.
- `dss_files`: maps each DSS to the set of files present on the disk, enabling efficient `delete_all` and failure cleanup.
- `write_block` and `write_recovered_block` update all indices atomically inside the lock, ensuring consistent state even with concurrent threads.
- `get_storage_stats` and `get_all_files` expose diagnostics used by validation scripts.

Using composite keys avoids nested dictionaries and simplifies deletions. All block payloads are stored as raw bytes so XOR reconstruction and parity checks remain efficient.

5.3 User Client

The user CLI encapsulates several responsibilities:

- Maintaining helper functions such as `compute_xor_parity`, `reconstruct_missing_block`, and `calculate_parity_disk`.
- Spawning per-stripe worker threads for copy/read/recovery, collecting results in shared arrays, and coordinating retries on parity failure.
- Translating filesystem operations into messages (e.g., reading files into stripes, padding, base64 encoding).
- Recording critical operations in logs so the validation scripts and grading steps can trace behavior.
- Managing recovery bookkeeping: determining which disk failed, orchestrating reads from surviving disks, reconstructing parity, and writing recovered blocks.

Thread lifetimes are carefully scoped so the user relinquishes the command port socket after each command finishes, preventing clashes with subsequent operations.

6 Design Decisions

Key design decisions include:

- Using a common message envelope for all messages.
- Enforcing critical sections for copy operations and ownership checks on `read`.
- Distributing parity with rotating XOR blocks so any single disk failure can be recovered.
- Keeping disk storage in memory with composite-key dictionaries for fast lookup and deletion.

7 Port Numbers

Because I am Group 198, the port numbers are 21200–21299 (inclusive).

8 Screenshots

```
parker@DESKTOP-69P9G1L:/mnt/c/Users/pconley1/Dropbox/1. Projects/CSE 434/socket-project/Group198$ git log --pret
ty=format:"%h - %an, %ad (Commit) - %cd (Author)"
23359c6 - parkerconley, Fri Sep 26 18:22:08 2025 -0700 (Commit) - Fri Sep 26 18:22:08 2025 -0700 (Author)
9e7a297 - parkerconley, Fri Sep 26 16:31:25 2025 -0700 (Commit) - Fri Sep 26 16:31:25 2025 -0700 (Author)
d5d8694 - parkerconley, Fri Sep 26 16:28:23 2025 -0700 (Commit) - Fri Sep 26 16:28:23 2025 -0700 (Author)
0e5176f - parkerconley, Fri Sep 26 15:39:36 2025 -0700 (Commit) - Fri Sep 26 15:39:36 2025 -0700 (Author)
177745a - parkerconley, Fri Sep 26 14:46:58 2025 -0700 (Commit) - Fri Sep 26 14:46:58 2025 -0700 (Author)
3a3cab2 - parkerconley, Fri Sep 26 13:57:43 2025 -0700 (Commit) - Fri Sep 26 13:57:43 2025 -0700 (Author)
90547a4 - parkerconley, Fri Sep 26 12:51:36 2025 -0700 (Commit) - Fri Sep 26 12:51:36 2025 -0700 (Author)
a545e6e - parkerconley, Thu Sep 25 19:16:22 2025 -0700 (Commit) - Thu Sep 25 19:16:22 2025 -0700 (Author)
db66c92 - parkerconley, Thu Sep 25 18:07:28 2025 -0700 (Commit) - Thu Sep 25 18:07:28 2025 -0700 (Author)
a4296eb - parkerconley, Thu Sep 25 17:59:53 2025 -0700 (Commit) - Thu Sep 25 17:59:53 2025 -0700 (Author)
e901420 - parkerconley, Thu Sep 25 16:13:18 2025 -0700 (Commit) - Thu Sep 25 16:13:18 2025 -0700 (Author)
```

Figure 1: `git log` output

I didn't include the GitHub commit history due to it being paywalled by GitHub Plus.

Figure 2: Repository commit history

```
parker@DESKTOP-69P9G1L:/mnt/c/Users/pconley1/Dropbox/1. Projects/CSE 434/socket-project/Group198$ git reflog
23359c6 (HEAD -> main) HEAD@{0}: commit: finished design doc
9e7a297 HEAD@{1}: commit: readme update
d5d8694 HEAD@{2}: commit: mostly finished
0e5176f HEAD@{3}: commit: added register-disk
177745a HEAD@{4}: commit: added user.py register-user functionality, seems to work after a brief test
3a3cab2 HEAD@{5}: commit: initial implementation of register-user in manager.py
90547a4 HEAD@{6}: commit: created design doc
a545e6e HEAD@{7}: commit: added design doc
db66c92 HEAD@{8}: commit: test
a4296eb HEAD@{9}: commit: initial file structure
e901420 (origin/main, origin/HEAD) HEAD@{10}: clone: from github.com:parconley/Group198.git
```

Figure 3: `git reflog` output

```
parker@DESKTOP-69P9G1L:/mnt/c/Users/pconley1/Dropbox/1. Projects/CSE 434/socket-project/Group198$ git fsck
Checking object directories: 100% (256/256), done.
Checking objects: 100% (3/3), done.
```

Figure 4: `git fsck` output

The figures above correspond to:

i. Output of: `git log --pretty=format:"%h - %an, %ad (Commit) - %cd (Author)"`
ii. Entire commit history of the GitHub repository (omitted due to paywall)
iii. Output of: `git reflog`
iv. Output of: `git fsck`

9 Video Demo Link and Timestamps

Accessible Link `https://youtu.be/xnzJNYWxVww`

Timestamps

1. (a): N/A - Didn't need to compile.
2. (b): 0:00 - You can see that I have a CloudLab VM and personal PC running.
3. (c): 0:00 - I setup the manager, disks, and users.
4. (d): 2:30 - I configure the DSS.
5. (e): 3:28 - I try to configure the DSS again, but it fails due to insufficient disks.
6. (f): 4:03 - I deregister the disks and users, though the disks cannot deregister becuase they are in use.

10 Extended Design for Full Project

For the next version of this project, I'll need to add commands, add threading, and record another video demo.